

CASE STUDY
192421226 - RAMYA S S

CSA0609 – DAA

Network Packet Analysis System (Sorting + Binary Search) A network monitoring system processes large volumes of packet data that must be sorted and searched efficiently. Analyze how sorting algorithms and Binary Search can be combined. Identify issues such as processing speed and data size. Apply divide and conquer techniques in both stages. Analyze overall time complexity and justify the solution.

Introduction

A network monitoring system handles a large volume of packet data that must be processed efficiently. To improve performance, sorting and searching techniques are applied. Sorting organizes packet data, while searching helps quickly locate specific packets.

Use of Sorting and Binary Search

Sorting Stage

- Packet data is first sorted using **Merge Sort**.
- Merge Sort follows the **Divide and Conquer** approach:
 - Divide data into smaller subarrays
 - Sort each subarray recursively
 - Merge them into a sorted list

Searching Stage

- After sorting, **Binary Search** is used.
- It repeatedly divides the search space into halves.
- Efficient for large datasets.

Divide and Conquer Approach

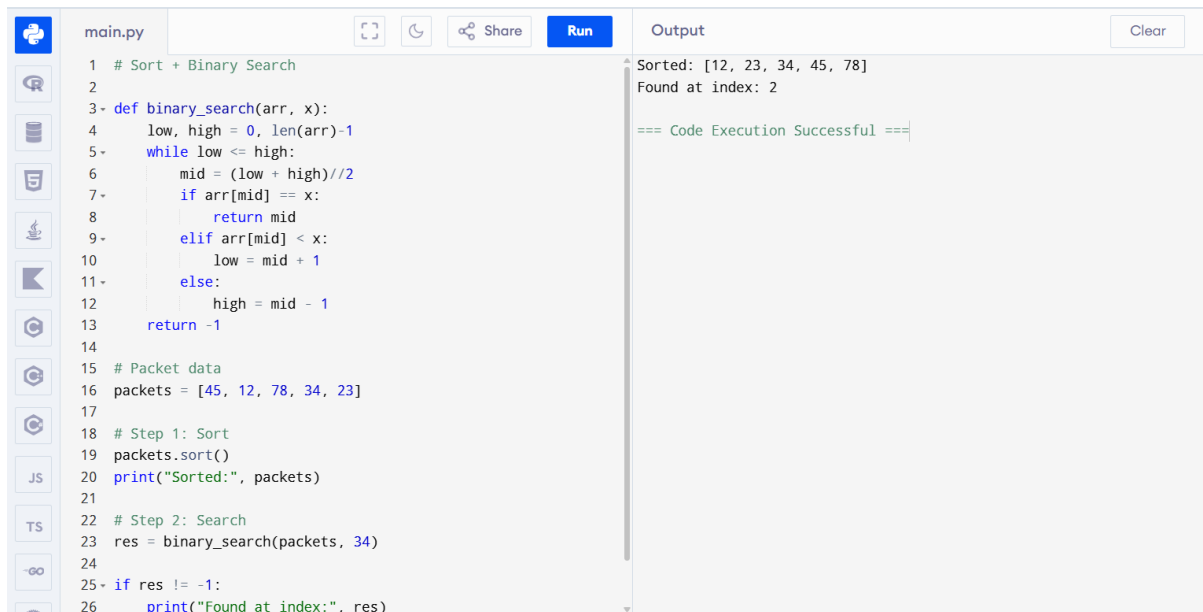
Merge Sort

- Divide → Split array into halves
- Conquer → Recursively sort both halves

- Combine → Merge sorted halves

Binary Search

- Divide → Find middle element
- Conquer → Search left or right half
- Repeat until element is found



The screenshot shows a code editor with a file named 'main.py'. The code implements a binary search function and applies it to a list of packet data. The output panel shows the sorted list and the index of the element 34.

```

1 # Sort + Binary Search
2
3 def binary_search(arr, x):
4     low, high = 0, len(arr)-1
5     while low <= high:
6         mid = (low + high)//2
7         if arr[mid] == x:
8             return mid
9         elif arr[mid] < x:
10            low = mid + 1
11        else:
12            high = mid - 1
13    return -1
14
15 # Packet data
16 packets = [45, 12, 78, 34, 23]
17
18 # Step 1: Sort
19 packets.sort()
20 print("Sorted:", packets)
21
22 # Step 2: Search
23 res = binary_search(packets, 34)
24
25 if res != -1:
26     print("Found at index:", res)
  
```

Output:

```

Sorted: [12, 23, 34, 45, 78]
Found at index: 2

=== Code Execution Successful ===
  
```

Issues in Network Packet Processing

1. Processing Speed

- Large packet data increases computation time
- Real-time systems require fast processing

2. Data Size

- Huge volume of packets
- Requires efficient memory usage

3. Scalability

- System must handle increasing traffic

Overall Complexity

- Total = Sorting + Searching
- $O(n \log n) + O(\log n) \approx O(n \log n)$

Justification of Solution

- Merge Sort ensures efficient sorting for large datasets
- Binary Search provides fast lookup
- Both use **Divide and Conquer**, improving performance
- Suitable for real-time network packet analysis

Conclusion

Combining Merge Sort and Binary Search enables efficient packet processing. This approach minimizes time complexity and improves system performance, making it ideal for handling large-scale network data.